

Librerie

```
1  #include <iostream>
2  #include <cstdlib>
3  #include <ctime>
4
5  using namespace std;
```

riga 1: include la libreria `<iostream>` che serve per prendere valori dall'utente in inserimento e per stampare valori all'utente in uscita

riga 2: include la libreria `<cstdlib>` che serve per usare le funzioni per generare un numero casuale (`rand()` e `srand()`)

riga 3: include la libreria `<ctime>` che serve per generare un valore casuale per la seed di `srand()`

riga 5: serve per poter usare gli elementi delle librerie senza dover scrivere prima `std::` :

Funzioni generali

```
7  void riempiVett(int v[], int dim) {
8      for (int i = 0; i < dim; i++) {
9          v[i] = rand() % 10 + 1;
10     }
11 }
12
13 void copiaVett(int v1[], int v2[], int dim) {
14     for (int i = 0; i < dim; i++) {
15         v2[i] = v1[i];
16     }
17 }
18
19 void stampaVett(int v[], int dim) {
20     for (int i = 0; i < dim; i++) {
21         cout << v[i] << "\t";
22     }
23     cout << endl;
24 }
```

riga 7: prototipo di `riempiVett()`

riga 8: itera per tutto il vettore

riga 9: assegna per ogni valore un valore casuale tra 1 e 10 (incluso)

riga 13: prototipo di `copiaVett()`

riga 14: itera per tutto il vettore

riga 15: copia il valore di un vettore su un altro

riga 19: prototipo di `stampaVett()`

riga 20: itera per tutto il vettore

riga 21: stampa gli elementi del vettore separati dal “\t”, che da un spaziamento uniforme

riga 23: salta una riga

Funzioni secondarie

```
26 void scambia(int& a, int& b, unsigned int& Asg) {
27     int c = a;
28     Asg++;
29     a = b;
30     Asg++;
31     b = c;
32     Asg++;
33 }
34
35 void shiftR(int v[], int dim, int i, int temporanea, unsigned int& Asg, unsigned int& Cft) {
36     int j = i - 1; //Iniziamo a guardare l'elemento precedente.
37     Asg++;
38     while ((j >= 0) && (v[j] > temporanea)) { //Finche' non siamo ancora arrivati all'estremo
39         Cft += 2;
40         v[j + 1] = v[j]; //sovrascriviamo la posizione di quello presente
41         Asg++;
42         //stampaVett(v, dim)
43         //v[j] = 0; //solo per sconi estetici
44         Asg++;
45         j--; //Facciamo un passo a sinistra per vedere se i
46         Asg++;
47     }
48     v[j + 1] = temporanea; //Quando non possiamo piu' shiftare gli elementi
49     Asg++;
50     stampaVett(v, dim);
51 }
52
```

riga 26: prototipo della funzione `scambia()`:

riga 27: appoggia il valore `a` su una variabile temporanea (`c`)

riga 29: `a` prende il valore di `b`

riga 31: `b` prende il valore di `a` tramite la variabile temporanea ©

riga 35: prototipo della funzione `shiftR()`:

riga 36: prende l'indice dell'elemento a sinistra

riga 38: finche' non siamo ancora arrivati all'estremo della sinistra e l'elemento sinistra attuale e' maggiore di quello presente...

riga 40: l'elemento a sinistra andra' nella posizione dell'elemento con cui stiamo lavorando

riga 45: spostiamoci a sinistra

riga 48: se siamo arrivati alla sinistra del vettore o l'elemento a sinistra e' piu' piccolo di quello che stiamo lavorando, si piazza l'elemento nella posizione rimasta

riga 50: stampa la situazione attuale del nostro vettore

Insertion sort

L'insertion sort simula l'ordinamento di carte in mano. Consiste nel prendere un elemento, e inserirlo nella posizione del vettore in cui i valori a sinistra sono minori dell'elemento e i valori a destra sono maggiori dell'elemento.

Serve anche un algoritmo che possa muovere l'elemento nella posizione in cui si vuole inserire il vettore e che possa spostare gli altri elementi indietro. Il shift right.

```
53 void insertionSort(int v[], int dim, unsigned int& Asg, unsigned int& Cft) {
54     for (int i = 1; i < dim; i++) { //Per ogni elemento a destra di quello a sinistra,
55         int temporanea = v[i]; //prendiamo il secondo su una variabile temporanea,
56         Asg++;
57         shiftR(v, dim, i, temporanea, Asg, Cft); //shift right
58     }
59 }
60
```

riga 53: prototipo della funzione `insertionSort()`

riga 54: itera su tutto il vettore tranne il primo elemento, perche' dal modo in cui e' strutturato l'algoritmo non serve andare sulla prima posizione, perche' essa sara' per forza l'elemento piu' piccolo

riga 55: crea una variabile temporanea

riga 57: chiama la funzione `shiftR()` per inserire l'elemento che abbiamo appena messo nella variabile temporanea a sinistra dell'elemento piu' piccolo possibile (l'ultimo elemento piu' grande di esso)

Selection sort

Il selection sort consiste nel trovare l'elemento piu' piccolo all'interno della parte del vettore non ancora ordinata e metterlo alla posizione successiva dell'ultimo elemento appena ordinato

```
61 void selectionSort(int v[], int dim, unsigned int& Asg, unsigned int& Cft) {
62     int i_min; //cerchiamo l'elemento minore.
63     Asg++;
64     for (int i = 0; i < dim - 1; i++) { //Per ogni elemento tranne l'ultimo,
65         i_min = i; //prendiamo prima quello piu' a sinistra
66         Asg++;
67         for (int j = i; j < dim; j++) { //Per i restanti a destra,
68             if (v[j] < v[i_min]) { //confrontiamolo con quello piu' a sinistra
69                 Cft++;
70                 i_min = j; //se e' piu' piccolo prendiamolo.
71                 Asg++;
72             }
73         }
74         scambia(v[i], v[i_min], Asg); //scambiamo il piu' piccolo con quello
75         stampaVett(v, dim);
76     }
77 }
78
```

riga 61: prototipo della funzione `selectionSort()`

riga 62: crea una variabile chiamata `i_min` che prendera' l'indice dell'elemento minore

riga 64: itera per tutto il vettore tranne l'ultimo elemento, perche' alla fine di tutte le iterazioni,

l'ultimo elemento sara' per forza quello piu' grande
 riga 65: la variabile che prendera' l'indirizzo dell'elemento piu' piccolo per ora prende quello dell'elemento piu' a sinistra (il secondo)
 riga 67: iteriamo sugli elementi non ancora ordinati
 riga 68-70: se uno degli elementi non ancora ordinati e' piu' piccolo di quello che e' presente su `v[i_min]`, allora `i_min` prendera' il suo indirizzo
 riga 74: dopo aver trovato l'elemento piu' piccolo tra gli elementi non ancora ordinati scambiamolo con la prima posizione che ha un elemento non ancora ordinato
 riga 75: stampa la situazione attuale del vettore

Bubble sort

Il bubble sort consiste in prendere un elemento che non sia in ordine e scambiarlo con gli elementi davanti finche' non incontra un elemento che sia maggiore di se stesso. Questi scambi devono succedere finche' non ci saranno piu' scambi. Cio' significa che il vettore e' del tutto ordinato.

```

79 void bubbleSort(int v[], int dim, unsigned int& Asg, unsigned int& Cft) {
80     bool scambio;                                //Controllo di scambi.
81     Asg++;
82     do {
83         Cft++;
84         scambio = false;                          //Per ora nessun scambio.
85         Asg++;
86         for (int i = 0; i < dim - 1; i++) { //Per ogni elemento tranne l'ult
87             if (v[i + 1] < v[i]) {              //se quello successivo e' minore
88                 Cft++;
89                 scambia(v[i], v[i + 1], Asg);    //scambiamoli.
90                 stampaVett(v, dim);
91                 scambio = true;                  //C'e' un scambio.
92                 Asg++;
93             }
94         }
95     } while (scambio);                          //Fai un'altra volta finche' ci
96     Cft--; //perche' il do-while esegue il codice almeno 1 volta
97 }                                                //(finche' il vettore non e' anc
98

```

riga 79: prototipo della funzione `bubbleSort()`
 riga 80: crea una variabile booleana chiamata `scambio` che dira' al programma se c'e' stato un scambio
 riga 82-95: esegui il blocco di codice finche' c'e' stato uno scambio
 riga 84: preimpostiamo `scambio` a `false`
 riga 86: iteriamo su tutti gli elementi tranne l'ultimo elemento, perche' alla fine di tutte le iterazioni, l'ultimo elemento sara' per forza quello piu' grande
 riga 87-91: se l'elemento a destra di quello che stiamo iterando adesso e' minore allora li scambieremo e in piu' stamperemo la situazione attuale del vettore

Main

```
99  int main() {
100      srand(time(NULL));
101
102      const int dim = 10;
103      int vInsertionSort[dim];
104      int vSelectionSort[dim];
105      int vBubbleSort[dim];
106
107      riempiVett(vInsertionSort, dim);
108      cout << "===Vettore iniziale===" << endl;
109      stampaVett(vInsertionSort, dim);
110      cout << endl;
111
112      copiaVett(vInsertionSort, vSelectionSort, dim);
113      copiaVett(vInsertionSort, vBubbleSort, dim);
114
115      unsigned int nAsgInsSort = 0;
116      unsigned int nAsgSelSort = 0;
117      unsigned int nAsgBubSort = 0;
118
119      unsigned int nCftInsSort = 0;
120      unsigned int nCftSelSort = 0;
121      unsigned int nCftBubSort = 0;
122
123      cout << "===Insertion sort===" << endl;
124      stampaVett(vInsertionSort, dim);
125      insertionSort(vInsertionSort, dim, nAsgInsSort, nCftInsSort);
126      cout << "Numero di assegnazioni: " << nAsgInsSort << endl << endl;
127      cout << "Numero di confronti: " << nCftInsSort << endl << endl;
128      cout << "Numero di operazioni: " << nAsgInsSort + nCftInsSort << endl << endl;
129
130      cout << "===Selection sort===" << endl;
131      stampaVett(vSelectionSort, dim);
132      selectionSort(vSelectionSort, dim, nAsgSelSort, nCftSelSort);
133      cout << "Numero di assegnazioni: " << nAsgSelSort << endl << endl;
134      cout << "Numero di confronti: " << nCftSelSort << endl << endl;
135      cout << "Numero di operazioni: " << nAsgSelSort + nCftSelSort << endl << endl;
136
137      cout << "===Bubble sort===" << endl;
138      stampaVett(vBubbleSort, dim);
139      bubbleSort(vBubbleSort, dim, nAsgBubSort, nCftBubSort);
140      cout << "Numero di assegnazioni: " << nAsgBubSort << endl << endl;
141      cout << "Numero di confronti: " << nCftBubSort << endl << endl;
142      cout << "Numero di operazioni: " << nAsgBubSort + nCftBubSort << endl << endl;
143  }
144
```

riga 99: prototipo della funzione `main()`

riga 100: genera una seed nuova ogni volta che si va ad eseguire il codice

riga 102: crea una variabile costante che si chiama `dim` per comunicare la dimensione del vettore

riga 103-105: creazione dei vettori da ordinare nella memoria, uno per ogni tipo di algoritmo di ordinamento

riga 107-113: riempimento/popoloamento dei 3 vettori e stampa della situazione iniziale dei vettori (che sono tutte 3 le stesse). Il primo e' stato generato casualmente con la funzione `riempiVett()`, gli altri 2 sono una copia del primo

riga 115-121: creazione di variabili di conteggio del numero di operazioni (assegnazioni e confronti), una per ogni algoritmo di ordinamento

riga 123-142: esecuzione degli algoritmi di ordinamento, stampa del vettore dopo ogni elemento ordinato e stampa dei dati di ogni algoritmo (numero di assegnazioni, numero dei confronti e numero di operazioni (num assegnazioni + num confronti))